

বেসিক Cryptography

প্রশ্ন: Cryptography কী?

উত্তর: Cryptography হলো information secure করার science। Plain text-কে cipher text-এ convert করে unauthorized access থেকে রক্ষা করা হয়।

প্রশ্ন: Cryptanalysis কী?

উত্তর: Key না জেনে cipher text থেকে plain text বের করার চেষ্টা। Brute force, frequency analysis ইত্যাদি cryptanalysis-এর method।

প্রশ্ন: Cryptology কী?

উত্তর: Cryptography এবং Cryptanalysis মিলিয়ে Cryptology। Secure communication তৈরি এবং ভাঙা দুটোই এর অংশ।

প্রশ্ন: Encryption এবং Decryption কী?

উত্তর: Encryption = plain text $\rightarrow$ cipher text। Decryption = cipher text $\rightarrow$ plain text। Key ব্যবহার করে উভয় operation হয়।

প্রশ্ন: Plain text এবং Cipher text কী?

উত্তর: Plain text = original readable message। Cipher text = encrypted unreadable message। যেমন "HELLO" $\rightarrow$ "KHOOR" (Caesar shift 3)।

প্রশ্ন: Key কী এবং কেন গুরুত্বপূর্ণ?

উত্তর: Key হলো encryption এবং decryption-এ ব্যবহৃত secret value। Key ছাড়া cipher text decode করা যায় না। Key-এর security-ই system-এর security।

প্রশ্ন: Symmetric Encryption কী?

উত্তর: Encryption এবং decryption-এ same key ব্যবহার হয়। Fast। যেমন DES, AES, Caesar, OTP। Key securely share করার সমস্যা আছে।

প্রশ্ন: Asymmetric Encryption কী?

উত্তর: Public key দিয়ে encrypt, private key দিয়ে decrypt। Slow কিন্তু key distribution সহজ।
যেমন RSA, ElGamal।

প্রশ্ন: Public Key এবং Private Key কী?

উত্তর: Public key সবাই জানতে পারে, encrypt করতে ব্যবহার হয়। Private key শুধু owner জানে, decrypt করতে ব্যবহার হয়।

প্রশ্ন: Substitution Cipher কী?

উত্তর: Letter replace হয় অন্য letter দিয়ে। Letter-এর position change হয় না। যেমন Caesar, Playfair, Monoalphabetic।

প্রশ্ন: Transposition Cipher কী?

উত্তর: Letter-এর position পরিবর্তন হয় কিন্তু letter নিজে change হয় না। Same letter ভিন্ন position-এ যায়।

প্রশ্ন: Block Cipher এবং Stream Cipher-এর পার্থক্য কী?

উত্তর: Block cipher fixed size block (যেমন 64-bit) একসাথে encrypt করে। DES block cipher। Stream cipher একটি একটি bit/byte encrypt করে। OTP stream cipher-এর মতো।

প্রশ্ন: Brute Force Attack কী?

উত্তর: সব possible key systematically try করা। Caesar-এ 26টি shift। AES-256-এ 2^{256} key — practically impossible।

প্রশ্ন: Frequency Analysis কী?

উত্তর: English-এ কিছু letter বেশি ব্যবহার হয় (e, t, a, o)। Cipher text-এ সবচেয়ে frequent letter সম্ভবত 'e'-র substitute। Monoalphabetic cipher-এ কাজ করে।

প্রশ্ন: Modular Arithmetic কী?

উত্তর: $(a + b) \% n$ । n -এর চেয়ে বড় না হয়ে remainder নেওয়া। Clock = mod 12। Cryptography-তে mod 26 (alphabet) বেশি ব্যবহার হয়।

প্রশ্ন: GCD কী?

উত্তর: Greatest Common Divisor। দুটি number-এর সবচেয়ে বড় common factor। $GCD(48, 18) = 6$ ।

প্রশ্ন: Coprime কী?

উত্তর: $GCD(a, b) = 1$ হলে a এবং b coprime বা relatively prime। RSA-এ e এবং $\phi(n)$ coprime হতে হয়।

প্রশ্ন: Modular Inverse কী?

উত্তর: $a \times a^{-1} \equiv 1 \pmod{m}$ । $GCD(a, m) = 1$ হলে exist করে। RSA-এ $d = e^{-1} \pmod{\phi(n)}$ বের করতে লাগে।

প্রশ্ন: Prime Number কেন cryptography-তে এত গুরুত্বপূর্ণ?

উত্তর: Large prime number factoring করা computationally hard। RSA, Diffie-Hellman, ElGamal সব prime-এর উপর based।

প্রশ্ন: Primitive Root কী?

উত্তর: g হলো prime p -এর primitive root যদি $g^1, g^2, \dots, g^{(p-1)} \pmod{p}$ সব 1 থেকে $p-1$ পর্যন্ত সব number exactly একবার generate করে।

প্রশ্ন: Discrete Logarithm Problem কী?

উত্তর: $g^x \pmod{p}$ জানা থাকলে x বের করা computationally hard। Diffie-Hellman এবং ElGamal এই problem-এর উপর secure।

প্রশ্ন: Confusion এবং Diffusion কী?

উত্তর: Confusion = key এবং cipher text-এর relationship complex করা (substitution)। Diffusion = plain text-এর statistical structure ছড়িয়ে দেওয়া (transposition)। Shannon এই concept দিয়েছেন।

প্রশ্ন: Kerckhoffs's Principle কী?

উত্তর: Cryptographic system-এর security শুধু key-এর secrecy-এর উপর depend করা উচিত, algorithm-এর secrecy-এর উপর না। Algorithm public হলেও system secure হওয়া উচিত।

প্রশ্ন: Man-in-the-Middle Attack কী?

উত্তর: Attacker দুটি party-র মাঝে বসে communication intercept করে। Diffie-Hellman MITM-এ vulnerable কারণ authentication নেই।

প্রশ্ন: Digital Signature কী?

উত্তর: Private key দিয়ে message sign করা। Public key দিয়ে verify করা যায়। Message authenticity এবং integrity নিশ্চিত করে।

Caesar Cipher (E1-1)

প্রশ্ন: Caesar Cipher কী?

উত্তর: Julius Caesar ব্যবহার করতেন। প্রতিটি letter-কে alphabet-এ নির্দিষ্ট shift-এ সরিয়ে encrypt করা। Shift=3 হলে A→D, B→E।

প্রশ্ন: Caesar Cipher-এর encrypt formula কী?

উত্তর: $C = (P - \text{base} + \text{shift}) \% 26 + \text{base}$ । Uppercase-এ base=65, Lowercase-এ base=97।

প্রশ্ন: Caesar Cipher-এর decrypt formula কী?

উত্তর: $P = (C - \text{base} - \text{shift}) \% 26 + \text{base}$ । অথবা $\text{decrypt}(\text{text}, \text{shift}) = \text{encrypt}(\text{text}, -\text{shift})$ ।

প্রশ্ন: "HELLO" shift=3 দিয়ে encrypt করলে কী হবে?

উত্তর: H(7)→K(10), E(4)→H(7), L(11)→O(14), L(11)→O(14), O(14)→R(17) | Cipher = "KHOOR" |

প্রশ্ন: "KHOOR" shift=3 দিয়ে decrypt করলে কী হবে?

উত্তর: K→H, H→E, O→L, O→L, R→O | Plain text = "HELLO" |

প্রশ্ন: Shift=0 দিলে কী হয়?

উত্তর: Cipher text = Plain text | কোনো encryption হয় না।

প্রশ্ন: Shift=26 দিলে কী হয়?

উত্তর: % 26 = 0 | Cipher text = Plain text | Shift=26 এবং Shift=0 same |

প্রশ্ন: Code-এ base = 65 কেন ব্যবহার হয়?

উত্তর: 'A'-এর ASCII value 65 | base বিয়োগ করলে A=0, B=1,...,Z=25 হয়। % 26 করলে 0-25 range-এ থাকে। তারপর base যোগ করলে আবার ASCII হয়।

প্রশ্ন: Lowercase-এ base কত?

উত্তর: 'a'-এর ASCII = 97 | তাই lowercase-এ base = 97 |

প্রশ্ন: c.isalpha() কেন check করা হয়?

উত্তর: Space, number, punctuation encrypt করার দরকার নেই। isalpha() শুধু letter encrypt করে, বাকি সব as-is রাখে।

প্রশ্ন: Caesar Cipher-এর key space কত?

উত্তর: মাত্র 26 | Shift 0-25 পর্যন্ত সম্ভব।

প্রশ্ন: Caesar Cipher-এর weakness কী কী?

উত্তর: মাত্র 26 possible key — brute force trivial | Frequency analysis কাজ করে | Chosen plain text attack-এ instantly break হয়।

প্রশ্ন: ROT13 কী?

উত্তর: Caesar cipher with shift=13 | ROT13 নিজেই নিজের inverse কারণ $13+13=26$ ।
 $ROT13(ROT13(text)) = text$ ।

প্রশ্ন: Vigenere Cipher Caesar থেকে কীভাবে ভালো?

উত্তর: Vigenere-এ প্রতিটি letter-এ ভিন্ন shift (key word অনুযায়ী)। Frequency analysis harder।
কিন্তু periodic key, তাই Kasiski test দিয়ে break করা যায়।

প্রশ্ন: Decrypt function-এ শুধু -shift কেন কাজ করে?

উত্তর: Encrypt করে shift +3 দিলে letter 3 এগিয়ে যায়। Decrypt করতে -3 shift দিলে 3 পিছিয়ে আসে। % 26 negative value-ও handle করে।

প্রশ্ন: "Python is great! Isn't it?" encrypt করলে punctuation কী হবে?

উত্তর: Punctuation যেমন !, ?, ' — unchanged থাকবে। শুধু alphabet shift হবে।

One-Time Pad (E1-2)

প্রশ্ন: One-Time Pad কী?

উত্তর: প্রতিটি encryption-এ random key ব্যবহার করা হয় যেটা message-এর সমান length।
Perfect secrecy দেয়। Theoretically unbreakable।

প্রশ্ন: OTP encrypt formula কী?

উত্তর: $C_i = (P_i + K_i) \% 26$ | P = plain text letter value (0-25), K = key letter value (0-25)।

প্রশ্ন: OTP decrypt formula কী?

উত্তর: $P_i = (C_i - K_i) \% 26$ ।

প্রশ্ন: OTP কেন theoretically unbreakable?

উত্তর: Key সম্পূর্ণ random এবং message-এর সমান length। Same cipher text ভিন্ন key দিয়ে যেকোনো plain text দিতে পারে। Attacker correct key জানার কোনো way নেই।

প্রশ্ন: Perfect Secrecy কী?

উত্তর: Claude Shannon দিয়েছেন। Cipher text থেকে plain text সম্পর্কে zero information পাওয়া যায়। OTP perfect secrecy দেয়।

প্রশ্ন: OTP-এর practical limitations কী?

উত্তর: Key message-এর সমান length হতে হবে। Key শুধু একবার ব্যবহার করতে হবে। Key securely share করা কঠিন। Key management complex।

প্রশ্ন: "Two-time pad" কী এবং কেন dangerous?

উত্তর: Same key দুবার ব্যবহার করলে $C1 \oplus C2 = P1 \oplus P2$ । Key cancel হয়ে যায়, plain text পাওয়া যায়।

প্রশ্ন: generate_key() কীভাবে কাজ করে?

উত্তর: random.choice(string.ascii_uppercase) দিয়ে random uppercase letter নেওয়া হয়। Message length-এর সমান random key তৈরি হয়।

প্রশ্ন: zip(text, key) কেন ব্যবহার হয়?

উত্তর: text এবং key-এর corresponding position-এর character pair করতে। ('H','K'), ('E','Y') ইত্যাদি pair তৈরি হয়।

প্রশ্ন: OTP-এ key কি reuse করা যাবে?

উত্তর: না। Key reuse করলে statistical analysis সম্ভব হয়। Key একবার ব্যবহার করতে হবে — এটাই "One-Time" মানে।

প্রশ্ন: OTP এবং Caesar Cipher-এর পার্থক্য কী?

উত্তর: Caesar-এ same shift সব letter-এ — easily breakable। OTP-এ প্রতিটি letter-এ আলাদা random shift — unbreakable। OTP = Vigenere with truly random key of same length।

প্রশ্ন: OTP-এ key কোথায় store করতে হবে?

উত্তর: Securely। Key compromise হলে message compromise হয়। Physical exchange (USB, paper) বা quantum key distribution ব্যবহার হয়।

Monoalphabetic Substitution Cipher (E1-3)

প্রশ্ন: Monoalphabetic Substitution Cipher কী?

উত্তর: প্রতিটি plain text letter-এ fixed unique substitute letter assign করা। A→X, B→Q, C→M ইত্যাদি। Fixed mapping throughout।

প্রশ্ন: Caesar এবং Monoalphabetic Cipher-এর পার্থক্য কী?

উত্তর: Caesar-এ সব letter same shift-এ সরে (shift 3 → A→D, B→E)। Monoalphabetic-এ প্রতিটি letter আলাদা random mapping — A→X, B→P ইত্যাদি।

প্রশ্ন: Monoalphabetic Cipher-এর key space কত?

উত্তর: $26! \approx 4.03 \times 10^{26}$ । Brute force practically impossible।

প্রশ্ন: generate_cipher() কী করে?

উত্তর: random.sample(string.ascii_uppercase, 26) দিয়ে 26টি letter random permutation তৈরি করে। dict(zip('A-Z', permutation)) দিয়ে mapping তৈরি।

প্রশ্ন: Decryption কীভাবে করা হয়?

উত্তর: Cipher mapping reverse করা হয়: reverse_cipher = {v: k for k, v in cipher.items()}।

Encrypted letter lookup করলে original letter পাওয়া যায়।

প্রশ্ন: Monoalphabetic Cipher-এর weakness কী?

উত্তর: Frequency analysis। English-এ 'e' সবচেয়ে বেশি (12.7%), তারপর 't' (9.1%), 'a' (8.2%)।

Cipher text-এ সবচেয়ে frequent letter সম্ভবত 'e'-র substitute।

প্রশ্ন: Monoalphabetic Cipher কি brute force-এ break হবে?

উত্তর: 26! key space-এ direct brute force impossible। কিন্তু frequency analysis, pattern matching দিয়ে break হয়।

প্রশ্ন: cipher.get(c, c) কী করে?

উত্তর: c letter cipher dictionary-তে থাকলে substitute return করে। না থাকলে c নিজেই return করে। Space, number, punctuation unchanged থাকে।

প্রশ্ন: Monoalphabetic-এ "the" কেন weakness তৈরি করে?

উত্তর: "the" English-এ সবচেয়ে common 3-letter word। Cipher text-এ same 3-letter pattern বারবার দেখলে সম্ভবত "the"-র substitute। Pattern recognition attack সম্ভব।

প্রশ্ন: Polyalphabetic Cipher কী এবং কেন ভালো?

উত্তর: Same letter ভিন্ন position-এ ভিন্ন cipher হতে পারে। Vigenere polyalphabetic। Frequency analysis কঠিন হয় কারণ same letter এক cipher দেয় না।

GCD — Euclidean Algorithm (E1-4)

প্রশ্ন: Euclidean Algorithm কীভাবে কাজ করে?

উত্তর: $\gcd(a, b) = \gcd(b, a \% b)$ । $b = 0$ হলে a হলো GCD। বারবার remainder নিতে থাকলে eventually 0 হয়।

প্রশ্ন: $\gcd(48, 18)$ step by step calculate করো।

উত্তর: $\gcd(48, 18)$: remainder= $48 \% 18 = 12 \rightarrow \gcd(18, 12)$: remainder= $18 \% 12 = 6 \rightarrow \gcd(12, 6)$: remainder= $12 \% 6 = 0 \rightarrow \gcd(6, 0)$: return 6। GCD=6।

প্রশ্ন: $\gcd(100, 75)$ কত?

উত্তর: $\gcd(100, 75) \rightarrow \gcd(75, 25) \rightarrow \gcd(25, 0) = 25$ | $GCD = 25$ |

প্রশ্ন: Euclidean Algorithm কেন efficient?

উত্তর: প্রতিটি step-এ number অনেক ছোট হয়। $O(\log(\min(a, b)))$ time complexity | Brute force $O(\min(a, b))$ |

প্রশ্ন: GCD Cryptography-তে কেন দরকার?

উত্তর: RSA-এ e এবং $\phi(n)$ coprime verify করতে। Extended Euclidean দিয়ে modular inverse বের করতে। Key generation-এ ব্যবহার।

প্রশ্ন: $a \% b$ কোথা থেকে আসে এবং কী মানে?

উত্তর: $a = bq + r$ (division algorithm) | $r = a \% b = \text{remainder}$ | $\gcd(a, b) = \gcd(b, r)$ কারণ a এবং b -এর common factor r -তেও থাকে।

প্রশ্ন: $\gcd(a, 0) = a$ কেন?

উত্তর: 0 সব number দিয়ে divisible ($0 = a \times 0$)। তাই $\gcd(a, 0) = a$ । Euclidean algorithm এখানে terminate হয়।

প্রশ্ন: While loop কীভাবে terminate হয়?

উত্তর: প্রতিটি iteration-এ b এর value কমে ($a \% b < b$ সবসময়)। Eventually $b = 0$ হয়, loop শেষ।

প্রশ্ন: Code-এ remainder variable কী করে?

উত্তর: $a \% b$ calculate করে store করে। তারপর $a = b$, $b = \text{remainder}$ করে next iteration-এর জন্য prepare করে।

Brute Force — Caesar (E1-5)

প্রশ্ন: Brute Force Attack কী?

উত্তর: সব possible key systematically try করে cipher decrypt করার চেষ্টা। Caesar-এ 0 থেকে 25 পর্যন্ত সব shift try করা।

প্রশ্ন: Caesar cipher-এ কেন brute force সহজ?

উত্তর: মাত্র 26টি possible shift। কম্পিউটার instant-এ সব try করতে পারে। Attacker manually readable text খুঁজে নেয়।

প্রশ্ন: brute_force() function কীভাবে কাজ করে?

উত্তর: shift 0 থেকে 25 পর্যন্ত loop করে প্রতিটি shift-এ decrypt করে print করে। Attacker যেটা readable সেটাই correct।

প্রশ্ন: Shift 0 দিলে output কী হয়?

উত্তর: Cipher text নিজেই output হয়। কারণ 0 shift মানে কোনো change নেই।

প্রশ্ন: Brute force attack কখন practical?

উত্তর: Key space ছোট হলে। Caesar = 26 key, trivial। DES = 2^{56} , কঠিন কিন্তু modern hardware-এ possible। AES-256 = 2^{256} , practically impossible।

প্রশ্ন: Brute force prevent করার উপায় কী?

উত্তর: Key space বড় করা (AES-256)। Rate limiting (too many wrong attempts block)। Account lockout। Key stretching (PBKDF2)।

প্রশ্ন: Automated brute force-এ attacker কীভাবে correct key চেনে?

উত্তর: Language detection (English words check), dictionary check, or specific known plain text pattern খোঁজে।

প্রশ্ন: Known Plain Text Attack কী?

উত্তর: Attacker কিছু plain text এবং corresponding cipher text জানে। এটা দিয়ে key বের করার চেষ্টা করে। Caesar-এ একটি letter জানলেই shift বের হয়।

প্রশ্ন: Chosen Plain Text Attack কী?

উত্তর: Attacker নিজে plain text choose করে encrypt করতে পারে এবং cipher text দেখতে পারে। Caesar-এ "A" encrypt করলে shift জানা যায়।

Playfair Cipher (E1-6)

প্রশ্ন: Playfair Cipher কী?

উত্তর: Digraph substitution cipher। একসাথে দুটি letter encrypt করা হয় 5×5 matrix ব্যবহার করে। 1854 সালে Charles Wheatstone তৈরি, Lord Playfair popularize করেন।

প্রশ্ন: Playfair 5×5 matrix কীভাবে তৈরি হয়?

উত্তর: Key-এর unique letter আগে (J=I), তারপর remaining alphabet (J বাদ) 5×5 grid fill করে। Key "MONARCHY" → M,O,N,A,R,C,H,Y তারপর B,D,E,F,G,I,K,L,P,Q,S,T,U,V,W,X,Z।

প্রশ্ন: Playfair-এ J কেন বাদ দেওয়া হয়?

উত্তর: 26 letter কিন্তু 5×5 = 25 cell। একটি letter বাদ দিতে হয়। এবং J phonetically similar, তাই J=I treat করা হয়।

প্রশ্ন: Playfair-এর তিনটি encryption rule কী?

উত্তর: Rule 1: Same row — প্রতিটি letter পরের column (circular)। Rule 2: Same column — প্রতিটি letter পরের row (circular)। Rule 3: Rectangle — একই row-এ other corner নেওয়া।

প্রশ্ন: Same letter pair (LL, AA) হলে কী করা হয়?

উত্তর: মাঝে 'X' ঢোকানো হয়। "BALLOON" → "BA LX LO ON"। Duplicate letter pair আলাদা করা হয়।

প্রশ্ন: Odd length text হলে কী করা হয়?

উত্তর: শেষে 'X' যোগ করা হয় padding হিসেবে। "HELLO" (5 letter) → "HE LX LO" (X added)।

প্রশ্ন: prepare_text() function কী করে?

উত্তর: Uppercase convert, J→I replace, space remove। Duplicate adjacent letter হলে মাঝে X ঢোকায়। Odd length হলে শেষে X যোগ করে। Pairs তৈরি করে।

প্রশ্ন: find_pos() function কী করে?

উত্তর: 5×5 matrix-এ একটি letter-এর row এবং column position বের করে (i, j) হিসেবে return করে।

প্রশ্ন: Same row encryption-এ % 5 কেন?

উত্তর: Column 4-এর পরে column 0-এ ফিরতে হয় (wrap around)। $(c+1) \% 5$ circular shift নিশ্চিত করে।

প্রশ্ন: Playfair decrypt এবং encrypt-এর পার্থক্য কী?

উত্তর: Encrypt-এ same row: +1 column, same col: +1 row। Decrypt-এ same row: -1 column, same col: -1 row। Rectangle rule same।

প্রশ্ন: Key "MONARCHY" দিয়ে "INSTRUMENTS" encrypt করলে কী হয়?

উত্তর: "GATLMZCLRQTX"। (Lab result থেকে)।

প্রশ্ন: Playfair কেন Caesar-এর চেয়ে secure?

উত্তর: Digraph encrypt করে single letter না। $26 \times 26 = 676$ possible digraph। Single letter frequency analysis কাজ করে না। Pair frequency analysis দরকার।

প্রশ্ন: Playfair-এর limitation কী?

উত্তর: Digraph frequency analysis possible। J এবং I distinguish করা যায় না। 'X' padding original message পরিবর্তন করে। Modern standard-এ insecure।

প্রশ্ন: Playfair কি symmetric?

উত্তর: হ্যাঁ। Same key দিয়ে encrypt এবং decrypt করা হয়।

Hill Cipher (E1-7)

প্রশ্ন: Hill Cipher কী?

উত্তর: 1929 সালে Lester Hill তৈরি। Matrix multiplication ব্যবহার করে block-based encryption। $n \times n$ key matrix দিয়ে n টি letter একসাথে encrypt।

প্রশ্ন: Hill Cipher encrypt formula কী?

উত্তর: $C = K \times P \pmod{26}$ । K = key matrix, P = plain text column vector। 2×2 matrix: $[C1, C2] = [[a, b], [c, d]] \times [P1, P2] \pmod{26}$ ।

প্রশ্ন: 2×2 Hill Cipher-এ "HE" encrypt কীভাবে হয়?

উত্তর: $H=7, E=4$ । $[C1, C2] = [[a, b], [c, d]] \times [7, 4]$ । $C1 = (a \times 7 + b \times 4) \% 26$, $C2 = (c \times 7 + d \times 4) \% 26$ ।

প্রশ্ন: Hill Cipher decrypt কীভাবে করা হয়?

উত্তর: Key matrix-এর modular inverse বের করতে হয়। $\text{Inverse_K} = (\text{det}^{-1} \times \text{adj}(K)) \% 26$ । $P = \text{Inverse_K} \times C \pmod{26}$ ।

প্রশ্ন: 2×2 matrix-এর adjugate কী?

উত্তর: $[[a, b], [c, d]]$ এর adjugate = $[[d, -b], [-c, a]]$ । Diagonal swap এবং off-diagonal নেগেটিভ।

প্রশ্ন: Determinant কীভাবে calculate হয়?

উত্তর: $\text{det}([[a, b], [c, d]]) = a \times d - b \times c$ ।

প্রশ্ন: Hill Cipher-এর key matrix valid হওয়ার শর্ত কী?

উত্তর: $\text{GCD}(\det(K), 26) = 1$ হতে হবে। না হলে modular inverse exist করে না, decryption possible না।

প্রশ্ন: Experiment E1-7-এ key $\begin{bmatrix} 3 & 3 \\ 2 & 5 \end{bmatrix}$ দিয়ে "HELP" encrypt করলে কী হয়?

উত্তর: HE: H=7, E=4 | $C1 = (3 \times 7 + 3 \times 4) \% 26 = (21 + 12) \% 26 = 33 \% 26 = 7 = H$ |

$C2 = (2 \times 7 + 5 \times 4) \% 26 = (14 + 20) \% 26 = 34 \% 26 = 8 = I$ | LP: similarly AT | Result: "HIAT" |

প্রশ্ন: mod_inverse() function কীভাবে কাজ করে?

উত্তর: 1 থেকে m-1 পর্যন্ত loop করে। $(a \times x) \% m == 1$ হলে x হলো modular inverse | Brute force approach |

প্রশ্ন: np.linalg.det() কেন integer round করা হয়?

উত্তর: Floating point calculation-এ det সামান্য error থাকতে পারে। $\text{int}(\text{round}(\dots))$ দিয়ে exact integer পাওয়া যায়।

প্রশ্ন: Hill Cipher কেন Playfair-এর চেয়ে complex?

উত্তর: Linear algebra (matrix multiplication) ব্যবহার করে। Key space অনেক বড়। $n \times n$ matrix-এ n টি letter একসাথে encrypt |

প্রশ্ন: Hill Cipher-এর weakness কী?

উত্তর: Known plain text attack-এ vulnerable | Enough plain text-cipher text pair জানলে matrix solve করা যায়।

Transposition Cipher (E1-8)

প্রশ্ন: Transposition Cipher কী?

উত্তর: Letter-এর position পরিবর্তন হয় কিন্তু letter নিজে change হয় না। Key অনুযায়ী column reorder করে encrypt করা হয়।

প্রশ্ন: Columnar Transposition Cipher কীভাবে কাজ করে?

উত্তর: Text-কে grid-এ row wise লেখা হয়। Key-এর alphabetical order অনুযায়ী column-wise পড়া হয়।

প্রশ্ন: Key "ZEBRA" দিয়ে column order কী হবে?

উত্তর: Z=5, E=2, B=1, R=4, A=0 (0-indexed alphabetical)। Column read order: A(col4), B(col2), E(col1), R(col3), Z(col0)।

প্রশ্ন: Padding 'X' কেন ব্যবহার হয়?

উত্তর: Text সব cell fill নাও করতে পারে। 'X' দিয়ে remaining cell fill করা হয় grid complete করতে। ljust() এই padding করে।

প্রশ্ন: sorted(range(cols), key=lambda k: key[k]) কী করে?

উত্তর: Key-এর character alphabetical order অনুযায়ী column index sort করে। এটাই column read order নির্ধারণ করে।

প্রশ্ন: Double encryption কেন করা হয়?

উত্তর: Single encryption-এর চেয়ে secure। Pattern আরো complex হয়, analysis কঠিন হয়। Same key দুবার apply।

প্রশ্ন: Experiment E1-8-এ "MEET ME AFTER PARTY" key "ZEBRA" দিয়ে double encrypt করলে কী হয়?

উত্তর: Single = "METXEFAXEAPXTTRXMERY"। Double = "XTXREMEMYTXTEFAXAPTR"।

প্রশ্ন: Transposition cipher frequency analysis-এ কেন survive করে?

উত্তর: Letter নিজে change হয় না, শুধু position change। Single letter frequency same থাকে। তবে bigram/trigram frequency পরিবর্তন হয়।

প্রশ্ন: Substitution এবং Transposition combine করলে কী সুবিধা?

উত্তর: উভয়ের weakness cover হয়। Substitution frequency confuse করে, Transposition pattern ভাঙে। Modern cipher (AES) উভয় technique ব্যবহার করে।

প্রশ্ন: Rail Fence Cipher কী?

উত্তর: একধরনের transposition cipher। Text zigzag pattern-এ multiple rail-এ লেখা হয়। Rail-wise পড়লে cipher text পাওয়া যায়।

ElGamal Cryptosystem (E1-9)

প্রশ্ন: ElGamal Cryptosystem কী?

উত্তর: 1985 সালে Taher ElGamal তৈরি করেন। Discrete logarithm problem-এর উপর based asymmetric encryption।

প্রশ্ন: ElGamal-এর public এবং private key কী?

উত্তর: Private key: x (random secret number)। Public key: (p, g, y) যেখানে $y = g^x \mod p$ ।

প্রশ্ন: $y = g^x \mod p$ — কেন x বের করা কঠিন?

উত্তর: Discrete Logarithm Problem। y এবং g জেনে x বের করা computationally hard বড় prime p -এর জন্য।

প্রশ্ন: ElGamal encrypt steps কী?

উত্তর: Random k choose (1 থেকে $p-2$)। $c_1 = g^k \mod p$ । $c_2 = m \times y^k \mod p$ । Cipher = (c_1, c_2) ।

প্রশ্ন: ElGamal decrypt steps কী?

উত্তর: $s = c_1^x \mod p$ । s^{-1} = modular inverse of s । $m = c_2 \times s^{-1} \mod p$ ।

প্রশ্ন: Decryption কেন correct?

উত্তর: $s = c_1^x = (g^k)^x = g^{(kx)} \mod p$ । $c_2 = m \times y^k = m \times g^{(xk)} \mod p$ । $c_2/s = m \times g^{(xk)} / g^{(xk)} = m$ ।

প্রশ্ন: ElGamal-এ random k কেন দরকার?

উত্তর: প্রতিটি encryption-এ different k মানে same message বারবার encrypt করলে different cipher। Probabilistic encryption — replay attack prevent করে।

প্রশ্ন: k একই রাখলে কী সমস্যা?

উত্তর: Same k দিয়ে দুটো message encrypt করলে c_1 same হয়। $c_{2_1}/c_{2_2} = m_1/m_2$ থেকে relation বের হয়। Security compromise।

প্রশ্ন: ElGamal এবং RSA-এর পার্থক্য কী?

উত্তর: RSA factoring problem-এ based, ElGamal discrete log problem-এ। ElGamal probabilistic (random k), RSA deterministic। ElGamal cipher twice message size।

প্রশ্ন: $\text{pow}(s, -1, p)$ কী করে?

উত্তর: Python 3.8+ এ s-এর modular inverse mod p বের করে। $s \times s^{-1} \equiv 1 \pmod{p}$ ।

DES (E2-1)

প্রশ্ন: DES কী?

উত্তর: Data Encryption Standard। 1977 সালে NIST standardize করে। Symmetric block cipher। 56-bit key, 64-bit block। Feistel network।

প্রশ্ন: DES-এ key কত bit?

উত্তর: Input 64-bit কিন্তু effective = 56-bit। প্রতি 8th bit parity check-এ ব্যবহার হয়।

প্রশ্ন: DES-এর block size কত?

উত্তর: 64-bit = 8 bytes। প্রতিটি block independently encrypt হয় (ECB mode-এ)।

প্রশ্ন: DES-এ কতটি round আছে?

উত্তর: 16টি round। প্রতিটি round-এ Feistel function apply হয়।

প্রশ্ন: Feistel Network কী?

উত্তর: Block-কে দুই ভাগে ভাগ করা। প্রতিটি round-এ একটি half একটি function-এ যায়, output XOR করে অন্য half-এর সাথে, তারপর swap। Reverse করলে decrypt হয়।

প্রশ্ন: ECB mode কী?

উত্তর: Electronic Code Book। প্রতিটি block independently encrypt। Same plain text block সবসময় same cipher text → pattern leakage। Simple কিন্তু insecure।

প্রশ্ন: DES-এ padding কেন দরকার?

উত্তর: Message 8-byte (64-bit) block-এর multiple না হলে last block incomplete। pad() extra bytes যোগ করে block complete করে। unpad() decrypt-এ remove করে।

প্রশ্ন: Experiment E2-1-এ key কত character?

উত্তর: Exactly 8 character = 64-bit। কম বা বেশি হলে ValueError আসে।

প্রশ্ন: enc.hex() কেন ব্যবহার হয়?

উত্তর: Encrypted bytes non-printable character হতে পারে। hex() দিয়ে hexadecimal string-এ convert করলে readable এবং printable হয়।

প্রশ্ন: DES কেন আজকাল insecure?

উত্তর: 56-bit key space ছোট। 1998 সালে EFF-এর "Deep Crack" machine 22 ঘণ্টায় DES break করে। AES (128/256-bit key) replace করেছে।

প্রশ্ন: Triple DES কী?

উত্তর: DES তিনবার apply। $C = DES_K3(DES_K2^{-1}(DES_K1(P)))$ । Effective key 112-bit বা 168-bit। DES-এর successor হিসেবে ব্যবহার হয়েছিল।

প্রশ্ন: AES কেন DES-এর চেয়ে ভালো?

উত্তর: AES key = 128/192/256-bit (DES = 56-bit) | Block size 128-bit (DES = 64-bit) | Faster এবং more secure | Current standard |

প্রশ্ন: S-Box কী?

উত্তর: Substitution Box | DES-এ 6-bit input → 4-bit output lookup table | Non-linearity আনে।
৪টি S-box DES-এর core |

RSA (E2-2)

প্রশ্ন: RSA কী?

উত্তর: 1977 সালে Rivest, Shamir, Adleman তৈরি করেন। Asymmetric encryption | Integer factorization problem-এর উপর based |

প্রশ্ন: RSA key generation steps কী?

উত্তর: দুটো large prime p, q | $n = p \times q$ | $\phi(n) = (p-1)(q-1)$ | $e: 1 < e < \phi(n), \text{GCD}(e, \phi(n))=1$ | $d = e^{-1} \pmod{\phi(n)}$ | Public = (e, n) , Private = (d, n) |

প্রশ্ন: RSA encrypt formula কী?

উত্তর: $C = M^e \pmod{n}$ | M = message (number), C = cipher text |

প্রশ্ন: RSA decrypt formula কী?

উত্তর: $M = C^d \pmod{n}$ |

প্রশ্ন: Experiment E2-2-এ $p=61, q=53$ দিলে n এবং $\phi(n)$ কত?

উত্তর: $n = 61 \times 53 = 3233$ | $\phi(n) = (61-1)(53-1) = 60 \times 52 = 3120$ |

প্রশ্ন: $e=17$ valid কেন?

উত্তর: $\text{GCD}(17, 3120) = 1$ | 17 এবং 3120 coprime | 17 prime এবং $3120 = 2^4 \times 3 \times 5 \times 13$ — 17 factor না।

প্রশ্ন: d কীভাবে calculate হয়?

উত্তর: $d = e^{-1} \bmod \phi(n) = 17^{-1} \bmod 3120 = 2753$ | Verification: $17 \times 2753 = 46801 = 15 \times 3120 + 1 \equiv 1 \pmod{3120}$ ।

প্রশ্ন: $M=65$ encrypt করলে cipher text কত?

উত্তর: $C = 65^{17} \bmod 3233 = 2790$ ।

প্রশ্ন: $C=2790$ decrypt করলে M কত?

উত্তর: $M = 2790^{2753} \bmod 3233 = 65$ ।

প্রশ্ন: $\phi(n) = (p-1)(q-1)$ কেন?

উত্তর: Euler's totient function | $n = p \times q$ হলে n -এর সাথে coprime সংখ্যা $= (p-1)(q-1)$ | p এবং q prime।

প্রশ্ন: e কীভাবে choose করা হয়?

উত্তর: $\text{GCD}(e, \phi(n)) = 1$ হতে হবে। Common choice: $e = 65537 (2^{16} + 1, \text{prime})$ | Fast exponentiation-এর জন্য convenient।

প্রশ্ন: $\text{pow}(e, -1, \phi)$ Python-এ কী করে?

উত্তর: Python 3.8+ এ modular inverse বের করে। Extended Euclidean Algorithm internally ব্যবহার করে।

প্রশ্ন: RSA কেন secure?

উত্তর: n থেকে p এবং q বের করা (factoring) large number-এ computationally infeasible | $n = 2048\text{-bit}$ হলে factor করা impossible আজকের computer-এ।

প্রশ্ন: RSA-এ small prime p এবং q কেন dangerous?

উত্তর: ছোট prime সহজে factor হয়। Experiment-এ $p=61$, $q=53$ শুধু educational purpose-এ। Real RSA-এ 1024-bit বা 2048-bit prime ব্যবহার হয়।

প্রশ্ন: RSA-তে e এবং d এর relationship কী?

উত্তর: $e \times d \equiv 1 \pmod{\phi(n)}$ । তাই $M^{(e \times d)} \equiv M \pmod{n}$ — Euler's theorem থেকে। Decrypt করলে original message ফিরে আসে।

প্রশ্ন: RSA কেন slow?

উত্তর: Large number exponentiation এবং modular reduction computationally expensive। Real application-এ RSA শুধু key exchange করে, তারপর symmetric (AES) ব্যবহার হয়।

প্রশ্ন: RSA digital signature কীভাবে কাজ করে?

উত্তর: Sender private key দিয়ে sign করে: $S = M^d \pmod{n}$ । Receiver public key দিয়ে verify: $M = S^e \pmod{n}$ । Private key শুধু sender-এর কাছে, তাই authentication নিশ্চিত।

Diffie-Hellman Key Exchange (E2-3)

প্রশ্ন: Diffie-Hellman কী?

উত্তর: 1976 সালে Whitfield Diffie এবং Martin Hellman তৈরি। Insecure channel-এ দুটি party secretly shared key establish করার method।

প্রশ্ন: DH শুধু key exchange করে, encrypt করে না কেন?

উত্তর: DH-এর output একটি shared secret number। সেই secret দিয়ে symmetric cipher (AES) key তৈরি করা হয়। Message encryption symmetric cipher করে।

প্রশ্ন: Diffie-Hellman-এর steps কী?

উত্তর: Public parameters: prime p , primitive root g । Alice private: a , public: $A = g^a \pmod{p}$ । Bob private: b , public: $B = g^b \pmod{p}$ । Shared: Alice = $B^a \pmod{p}$, Bob = $A^b \pmod{p}$ ।

প্রশ্ন: কেন Alice এবং Bob same shared key পায়?

উত্তর: Alice key = $B^a = (g^b)^a = g^{(ab)} \bmod p$ | Bob key = $A^b = (g^a)^b = g^{(ab)} \bmod p$ |

উভয়ই $g^{(ab)} \bmod p$ |

প্রশ্ন: Experiment E2-3-এ $p=7$, $g=3$, $a=4$, $b=5$ দিলে A, B কত?

উত্তর: $A = 3^4 \bmod 7 = 81 \bmod 7 = 4$ | $B = 3^5 \bmod 7 = 243 \bmod 7 = 5$ |

প্রশ্ন: Shared key কত ছিল?

উত্তর: Alice: $B^a \bmod p = 5^4 \bmod 7 = 625 \bmod 7 = 2$ | Bob: $A^b \bmod p = 4^5 \bmod 7 = 1024 \bmod 7 = 2$ | Both = 2 |

প্রশ্ন: Attacker A এবং B জানলেও shared key বের করতে পারবে না কেন?

উত্তর: $A = g^a \bmod p$ জেনে a বের করতে হবে — Discrete Logarithm Problem | Large p-এর জন্য computationally infeasible |

প্রশ্ন: DH কোন attack-এ vulnerable?

উত্তর: Man-in-the-Middle (MITM) attack | Eve Alice এবং Bob-এর মাঝে বসে দুটো আলাদা DH exchange করতে পারে | Authentication নেই বলে DH vulnerable |

প্রশ্ন: MITM prevent করার উপায়?

উত্তর: Certificate Authority (CA) দিয়ে public key authenticate করা | Digital signature ব্যবহার করা | PKI (Public Key Infrastructure) |

প্রশ্ন: Primitive Root g কেন গুরুত্বপূর্ণ?

উত্তর: g^1 থেকে $g^{(p-1)} \bmod p$ সব 1 থেকে $p-1$ generate করে | এতে exponent থেকে base guess করা আরো কঠিন হয় |

প্রশ্ন: p ছোট হলে DH কেন insecure?

উত্তর: ছোট p -এর জন্য discrete log সহজে calculate করা যায়। Real DH-এ $p = 2048$ -bit prime বা তার বেশি ব্যবহার হয়।

প্রশ্ন: ECDH কী?

উত্তর: Elliptic Curve Diffie-Hellman। Elliptic curve group-এ DH করা। Same security much smaller key size দেয়। Modern TLS-এ ব্যবহার হয়।

Modular Arithmetic (E2-4)

প্রশ্ন: Modular Arithmetic কী?

উত্তর: Number-কে fixed modulus দিয়ে ভাগ করে remainder নিয়ে কাজ করা। $a \bmod n$ = remainder when a divided by n । Clock = mod 12।

প্রশ্ন: $(a + b) \bmod p$ কীভাবে calculate হয়?

উত্তর: a এবং b যোগ করে p দিয়ে ভাগ করে remainder। $(17 + 8) \bmod 7 = 25 \bmod 7 = 4$ ।

প্রশ্ন: $(a - b) \bmod p$ negative হলে?

উত্তর: Python-এ % operator negative result automatically positive করে। $(2-5) \bmod 7 = -3 \bmod 7 = 4$ । Mathematically: $-3 + 7 = 4$ ।

প্রশ্ন: Modular Multiplication কীভাবে কাজ করে?

উত্তর: $(a \times b) \bmod p$ । যেমন $(5 \times 6) \bmod 7 = 30 \bmod 7 = 2$ ।

প্রশ্ন: $\text{pow}(g, a, p)$ Python-এ কী করে?

উত্তর: $g^a \bmod p$ efficiently calculate করে। Fast modular exponentiation (Square-and-Multiply) algorithm ব্যবহার করে। $O(\log a)$ multiplications।

প্রশ্ন: Primitive Root verify কীভাবে করা যায়?

উত্তর: g^1 থেকে $g^{(p-1)} \bmod p$ calculate করলে 1 থেকে $p-1$ সব value exactly একবার আসলে g হলো primitive root।

প্রশ্ন: Experiment E2-4-এ $g=3$, $p=7$ primitive root কিনা verify করো।

উত্তর: $3^1=3$, $3^2=2$, $3^3=6$, $3^4=4$, $3^5=5$, $3^6=1$ (all mod 7)। সব 1-6 একবার এসেছে। তাই $g=3$ হলো $p=7$ -এর primitive root।

প্রশ্ন: Order of element কী?

উত্তর: $g^k \equiv 1 \pmod{p}$ হওয়ার সবচেয়ে ছোট positive k । Primitive root-এর order = $p-1$ ।

প্রশ্ন: Fermat's Little Theorem কী?

উত্তর: p prime এবং $\text{GCD}(a, p) = 1$ হলে: $a^{(p-1)} \equiv 1 \pmod{p}$ । RSA decryption proof-এ ব্যবহার হয়।

প্রশ্ন: Modular Exponentiation কেন important?

উত্তর: RSA, DH, ElGamal সব $M^e \bmod n$ calculate করে। Large exponent-এ naive multiplication impossible। Square-and-multiply efficient।

Extended Euclidean Algorithm (E2-5)

প্রশ্ন: Extended Euclidean Algorithm কী?

উত্তর: Regular Euclidean GCD-এর extension। GCD বের করার পাশাপাশি x এবং y বের করে যেখানে: $ax + by = \text{GCD}(a, b)$ ।

প্রশ্ন: Bezout's Identity কী?

উত্তর: For any a, b : $ax + by = \text{GCD}(a, b)$ এমন integer x, y সবসময় exist করে। Extended Euclidean এই x, y বের করে।

প্রশ্ন: Extended Euclidean কীভাবে modular inverse দেয়?

উত্তর: $\text{GCD}(a, m) = 1$ হলে: $a \cdot x + m \cdot y = 1 \mid \text{Mod } m$ নিলে: $a \cdot x \equiv 1 \pmod{m}$ । তাই x হলো a -এর modular inverse।

প্রশ্ন: Recursive function কীভাবে কাজ করে?

উত্তর: Base case: $b=0$ হলে $\text{gcd}=a$, $x=1$, $y=0$ । Recursive: $\text{gcd}(b, a \% b)$ call করে $x1, y1$ পায়। তারপর $x = y1$, $y = x1 - (a // b) \cdot y1$ ।

প্রশ্ন: Verification formula কী?

উত্তর: $a \cdot x + b \cdot y = \text{GCD}(a, b)$ । Code-এ: `print(f"{a}/{x} + {b}/{y} =", a*x + b*y)`।

প্রশ্ন: Extended Euclidean দিয়ে 17-এর modular inverse mod 3120 বের করো।

উত্তর: `extended_gcd(17, 3120) → gcd=1, x=2753`। Verification: $17 \times 2753 = 46801 = 15 \times 3120 + 1 \equiv 1 \pmod{3120}$ । তাই $17^{-1} \pmod{3120} = 2753$ ।

প্রশ্ন: Modular inverse কখন exist করে না?

উত্তর: $\text{GCD}(a, m) \neq 1$ হলে exist করে না। যেমন 4 এবং 6: $\text{GCD}(4, 6) = 2 \neq 1$, তাই $4^{-1} \pmod{6}$ exist করে না।

প্রশ্ন: Regular Euclidean এবং Extended Euclidean-এর পার্থক্য কী?

উত্তর: Regular শুধু GCD দেয়। Extended GCD + Bezout coefficients (x, y) দেয়। Extended RSA-এর d calculate করতে ব্যবহার হয়।

প্রশ্ন: Extended Euclidean-এর time complexity কত?

উত্তর: Regular Euclidean-এর মতো $O(\log(\min(a, b)))$ । Same recursion depth, just extra coefficient tracking।

তুলনামূলক প্রশ্ন

প্রশ্ন: Caesar, Playfair, Hill Cipher-এর complexity তুলনা করো।

উত্তর: Caesar: single letter, shift only, 26 key | Playfair: digraph, 5×5 matrix, more complex | Hill: matrix multiplication, largest key space | Complexity: Caesar < Playfair < Hill |

প্রশ্ন: Symmetric (DES) এবং Asymmetric (RSA)-এর কোনটি কখন ব্যবহার করবো?

উত্তর: DES/AES fast — bulk data encryption-এ। RSA slow — key exchange এবং digital signature-এ। Real system: RSA দিয়ে AES key exchange, তারপর AES দিয়ে data encrypt |

প্রশ্ন: RSA, Diffie-Hellman, ElGamal-এর mathematical basis কী?

উত্তর: RSA = Integer Factorization Problem | Diffie-Hellman = Discrete Logarithm Problem | ElGamal = Discrete Logarithm Problem | তিনটিই hard mathematical problem-এর উপর secure |

প্রশ্ন: OTP এবং DES-এর পার্থক্য কী?

উত্তর: OTP theoretically unbreakable কিন্তু impractical (key management) | DES practical কিন্তু brute force possible (56-bit key) | OTP = perfect secrecy, DES = computational security |

প্রশ্ন: Classical cipher (Caesar, Playfair) এবং Modern cipher (DES, RSA)-এর পার্থক্য কী?

উত্তর: Classical cipher manually doable, small key space, easily breakable | Modern cipher computer-based, large key space, mathematically secure | Classical = education, Modern = real-world |

প্রশ্ন: Transposition এবং Substitution combine কেন করা হয়?

উত্তর: Substitution: letter replace — frequency confuse করে। Transposition: position change — pattern break করে। দুটো combine করলে Shannon-এর confusion+diffusion উভয় পাওয়া যায়। AES এই approach follow করে।

প্রশ্ন: Hash Function এবং Encryption-এর পার্থক্য কী?

উত্তর: Encryption reversible — decrypt করা যায়। Hash function one-way — original বের করা infeasible | Hash integrity check-এ, encryption confidentiality-তে ব্যবহার হয়।

প্রশ্ন: Perfect Forward Secrecy কী?

উত্তর: প্রতিটি session-এ নতুন temporary key ব্যবহার। Long-term key compromise হলেও past session secure থাকে। Ephemeral DH (DHE) ব্যবহার করে achieve করা হয়।

প্রশ্ন: Quantum Computer Cryptography-তে কী threat?

উত্তর: Shor's Algorithm RSA এবং DH break করতে পারে। Grover's Algorithm symmetric key brute force faster করে। Post-quantum cryptography (lattice-based) develop হচ্ছে।

প্রশ্ন: TLS/HTTPS কোন algorithms ব্যবহার করে?

উত্তর: Key exchange: ECDHE (Elliptic Curve DH) | Authentication: RSA or ECDSA certificate |
Symmetric encryption: AES-256-GCM | MAC: HMAC-SHA256 |

শুভকামনা ভাইভায়! 🍀🔥